

✓ Ejercicio de programación Regresión Polinomial

```
from google.colab import drive
drive.mount("/content/gdrive")
```

Drive already mounted at /content/gdrive; to attempt to forcibly remount, call

```
# utilizado para manejos de directorios y rutas
import os

# Computacion vectorial y cientifica para python
import numpy as np

# Librerias para graficación (trazado de gráficos)
from matplotlib import pyplot
from mpl_toolkits.mplot3d import Axes3D # Necesario para graficar superficies

# llama a matplotlib a embeber graficas dentro de los cuadernillos
%matplotlib inline
```

✓ 2 Regresión polinómica

Se implementa la regresion polinomial para predecir el la capacidad adquisitiva de una persona. El archivo `Datasets/capacidad_adquisitiva.csv` contiene un dataset para entrenamiento de capacidad adquisitiva de las personas considerando la edad de las personas de la ciudad de Sucre. La primera columna es la edad y la segunda columna es la capacidad adquisitiva en bolivianos por mes.

2.1 Normalización de características

Al visualizar los datos se puede observar que las características tienen diferentes magnitudes, por lo cual se debe transformar cada valor en una escala de valores similares, esto con el fin de que el descenso por el gradiente pueda converger mas rapidamente.

```
# Cargar datos
#data = np.loadtxt(os.path.join('Datasets', 'capacidad_adquisitiva.csv'), del
#from numpy import genfromtxt
#data = genfromtxt(os.path.join('Datasets', 'capacidad_adquisitiva.csv'), del
data = np.loadtxt('/content/gdrive/MyDrive/Colab Notebooks/machine learning/c
#print(data)
X = data[:, :1]
y = data[:, 1]
m = y.size
#print(X)
#print(y)
```

```
# imprimir algunos puntos de datos
#print('{:>8s}{:>10s}'.format(X, y))
#print('- '*26)
for i in range(20):
    print('{:8.0f}{:10.0f}'.format(X[i, 0], y[i]))
```

```
3      1
4      2
7      3
15     5
17    10
20    50
25   500
27   800
30  6000
33  8000
38 12000
40 15000
45 18000
50 25000
55 23000
60 20000
65  9000
70  3800
75  3500
80  3000
```

La desviación estándar es una forma de medir cuánta variación hay en el rango de valores de una característica en particular (la mayoría de los puntos caeran en un rango de ± 2 en relación a la desviaciones estándar de la media); esta es una alternativa a tomar el rango de valores (max-min). En `numpy`, se puede usar la función `std` para calcular la desviacion estandar.

Por ejemplo, la característica `X[:, 0]` contiene todos los valores de x_1 (edades) en el conjunto de entrenamiento, entonces `np.std(X[:, 0])` calcula la desviacion estandar de las edades. En el momento en que se llama a la función `featureNormalize`, la columna adicional de unos correspondiente a $x_0 = 1$ aún no se ha agregado a X .

****Notas para la implementación:**** Cuando se normalize una característica, es importante almacenar los valores usados para la normalización - el valor de la media y el valor de la desviación estandar usado para los calculos. Despues de aprender los parametros del modelo, se deseara predecir la capacidad adquisitiva que no se han visto antes. Dado un nuevo valor de x (edad), primero se debe normalizar x usando la media y la desviacion estandar que se empleo anteriormente en el conjunto de entrenamiento para entrenar el modelo.

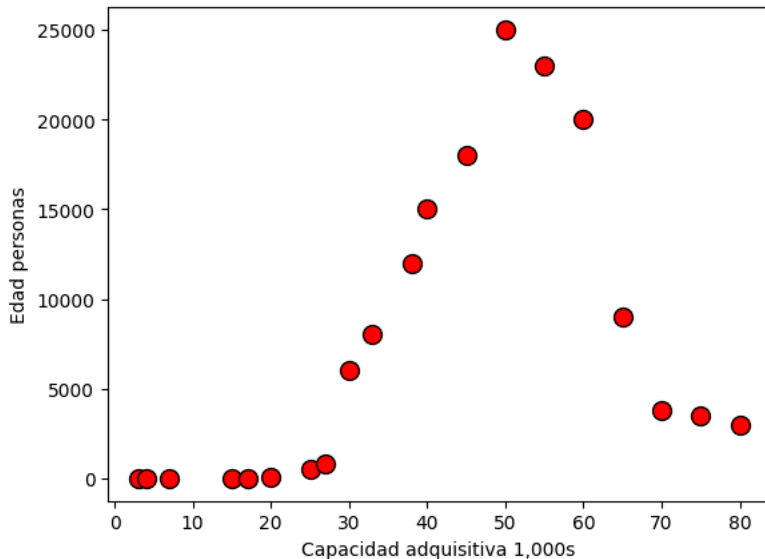
```
def plotData(x, y):
    #Grafica los puntos x e y en una figura nueva.

    fig = pyplot.figure() # abre una nueva figura

    pyplot.plot(x, y, 'ro', ms=10, mec='k')
    pyplot.ylabel('Edad personas')
```

```
pyplot.xlabel('Capacidad adquisitiva 1,000s')
```

```
plotData(X, y)
```



```
X = np.concatenate([X, X * X], axis=1)
```

```
print(X)
```

```
[[3.000e+00 9.000e+00]
 [4.000e+00 1.600e+01]
 [7.000e+00 4.900e+01]
 [1.500e+01 2.250e+02]
 [1.700e+01 2.890e+02]
 [2.000e+01 4.000e+02]
 [2.500e+01 6.250e+02]
 [2.700e+01 7.290e+02]
 [3.000e+01 9.000e+02]
 [3.300e+01 1.089e+03]
 [3.800e+01 1.444e+03]
 [4.000e+01 1.600e+03]
 [4.500e+01 2.025e+03]
 [5.000e+01 2.500e+03]
 [5.500e+01 3.025e+03]
 [6.000e+01 3.600e+03]
 [6.500e+01 4.225e+03]
 [7.000e+01 4.900e+03]
 [7.500e+01 5.625e+03]
 [8.000e+01 6.400e+03]]
```

```
def featureNormalize(X):
    X_norm = X.copy()
```

```

mu = np.zeros(X.shape[1])
sigma = np.zeros(X.shape[1])

mu = np.mean(X, axis = 0)
sigma = np.std(X, axis = 0)
X_norm = (X - mu) / sigma

return X_norm, mu, sigma

```

```

# llama featureNormalize con los datos cargados
X_norm, mu, sigma = featureNormalize(X)

#print(X)
#print('Media calculada:', mu)
#print('Desviación estandar calculada:', sigma)
#print(X_norm)

```

```
print(X_norm)
```

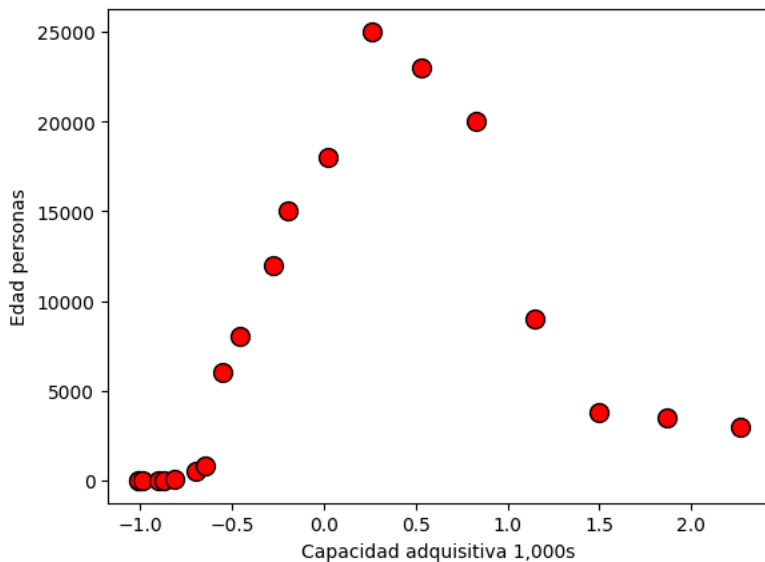
```

[[-1.49909249e+00 -1.01251109e+00]
 [-1.45620000e+00 -1.00892198e+00]
 [-1.32752253e+00 -9.92001936e-01]
 [-9.84382623e-01 -9.01761677e-01]
 [-8.98597645e-01 -8.68947037e-01]
 [-7.69920178e-01 -8.12034147e-01]
 [-5.55457733e-01 -6.96670180e-01]
 [-4.69672755e-01 -6.43346390e-01]
 [-3.40995288e-01 -5.55669775e-01]
 [-2.12317821e-01 -4.58764043e-01]
 [ 2.14462445e-03 -2.76745339e-01]
 [ 8.79296025e-02 -1.96759655e-01]
 [ 3.02392048e-01  2.11500607e-02]
 [ 5.16854493e-01  2.64696214e-01]
 [ 7.31316938e-01  5.33878804e-01]
 [ 9.45779383e-01  8.28697831e-01]
 [ 1.16024183e+00  1.14915330e+00]
 [ 1.37470427e+00  1.49524520e+00]
 [ 1.58916672e+00  1.86697354e+00]
 [ 1.80362916e+00  2.26433831e+00]]

```

Despues de `featureNormalize` la funcion es provada, se añade el temino de interseccion a `X_norm`:

```
plotData(X_norm[:,1], y)
```



```
# Añade el termino de interseccion a X
# (Columna de unos para X0)
#X_norm = np.concatenate([X_norm, X_norm * X_norm], axis=1)
X = np.concatenate([np.ones((m, 1)), X_norm], axis=1)
```

```
print(X)
```

```
[[ 1.00000000e+00 -1.49909249e+00 -1.01251109e+00]
 [ 1.00000000e+00 -1.45620000e+00 -1.00892198e+00]
 [ 1.00000000e+00 -1.32752253e+00 -9.92001936e-01]
 [ 1.00000000e+00 -9.84382623e-01 -9.01761677e-01]
 [ 1.00000000e+00 -8.98597645e-01 -8.68947037e-01]
 [ 1.00000000e+00 -7.69920178e-01 -8.12034147e-01]
 [ 1.00000000e+00 -5.55457733e-01 -6.96670180e-01]
 [ 1.00000000e+00 -4.69672755e-01 -6.43346390e-01]
 [ 1.00000000e+00 -3.40995288e-01 -5.55669775e-01]
 [ 1.00000000e+00 -2.12317821e-01 -4.58764043e-01]
 [ 1.00000000e+00 2.14462445e-03 -2.76745339e-01]
 [ 1.00000000e+00 8.79296025e-02 -1.96759655e-01]
 [ 1.00000000e+00 3.02392048e-01 2.11500607e-02]
 [ 1.00000000e+00 5.16854493e-01 2.64696214e-01]
 [ 1.00000000e+00 7.31316938e-01 5.33878804e-01]
 [ 1.00000000e+00 9.45779383e-01 8.28697831e-01]
 [ 1.00000000e+00 1.16024183e+00 1.14915330e+00]
 [ 1.00000000e+00 1.37470427e+00 1.49524520e+00]
 [ 1.00000000e+00 1.58916672e+00 1.86697354e+00]
 [ 1.00000000e+00 1.80362916e+00 2.26433831e+00]]
```

▼ 2.2 Descenso por el gradiente

En el ejemplo anterior se implemento el descenso por el gradiente para un problema de regresion univariable. La unica diferencia es que ahora existe una caracteristica adicional en la matriz X . La función de hipótesis y la regla de actualización del descenso del gradiente por lotes permanecen sin cambios.

La implementacion de las funciones `computeCostMulti` y `gradientDescentMulti` son similares a la funcion de costo y función de descenso por el gradiente de la regresión lineal multiple es similar al de la regresion lineal multivariable. Es importante garantizar que el codigo soporte cualquier numero de caracteristicas y esten bien vectorizadas.

Se puede utilizar `shape`, propiedad de los arrays `numpy`, para identificar cuantas caracteristicas estan consideradas en el dataset.

****Nota de implementación:**** En el caso de multivariantes, la función de costo puede se escrita considerando la forma vectorizada de la siguiente manera:

$$J(\theta) = \frac{1}{2m}(X\theta - \vec{y})^T(X\theta - \vec{y})$$

donde:

$$X = \begin{pmatrix} -(x^{(1)})^T - \\ -(x^{(2)})^T - \\ \vdots \\ -(x^{(m)})^T - \end{pmatrix} \quad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

La version vectorizada es eficiente cuando se trabaja con herramientas de calculo numericos computacional como `numpy`.

```
def computeCostMulti(X, y, theta):
    # Inicializa algunos valores utiles
    m = y.shape[0] # numero de ejemplos de entrenamiento

    J = 0

    h = np.dot(X, theta)

    J = (1/(2 * m)) * np.sum(np.square(np.dot(X, theta) - y))

    return J
```

```
def gradientDescentMulti(X, y, theta, alpha, num_iters):

    # Inicializa algunos valores
    m = y.shape[0] # numero de ejemplos de entrenamiento

    # realiza una copia de theta, el cual será acutalizada por el descenso pc
    theta = theta.copy()
```

```
J_history = []

for i in range(num_iters):
    theta = theta - (alpha / m) * (np.dot(X, theta) - y).dot(X)
    J_history.append(computeCostMulti(X, y, theta))

return theta, J_history
```

3.2.1 Seleccionando coeficientes de aprendizaje

```
# Elegir algun valor para alpha (probar varias alternativas)
alpha = 0.001
num_iters = 100000

# inicializa theta y ejecuta el descenso por el gradiente
theta = np.zeros(3)
theta, J_history = gradientDescentMulti(X, y, theta, alpha, num_iters)

# Grafica la convergencia del costo
pyplot.plot(np.arange(len(J_history)), J_history, lw=2)
pyplot.xlabel('Numero de iteraciones')
pyplot.ylabel('Costo J')

# Muestra los resultados del descenso por el gradiente
print('theta calculado por el descenso por el gradiente: {s}'.format(str(theta)))

# La capacidad adquisitiva de una persona de 33 años
X_array = [1, 34, 1156]
X_array[1:3] = (X_array[1:3] - mu) / sigma
price = np.dot(X_array, theta) # Se debe cambiar esto

print('La capacidad adquisitiva para una persona de 33 (usando el descenso por
```

```
plotData(X[:, 1], y)
pyplot.plot(X[:, 1], np.dot(X, theta), '-')
```

```
X_array = [1, 23, 529]
X_array[1:3] = (X_array[1:3] - mu) / sigma
```

```
X_array[1:3]
print(np.dot(X_array, theta))
```

2.3 Ecuacion de la Normal

Una manera de calcular rapidamente el modelo de una regresion lineal es:

$$\theta = (X^T X)^{-1} X^T \vec{y}$$

Utilizando esta formula no requiere que se escale ninguna característica, y se obtendrá una solución exacta con un solo cálculo: no hay "bucles de convergencia" como en el descenso por el gradiente.

Primero se recargan los datos para garantizar que las variables no estén modificadas. Recordar que no es necesario escalar las características, se debe agregar la columna

```
# Cargar datos
data = np.loadtxt('/content/gdrive/MyDrive/Colab Notebooks/machine learning/c
X = data[:, :1]
y = data[:, 1]
m = y.size
X_original = X.copy()
X = np.concatenate([X, X * X], axis=1)
X = np.concatenate([np.ones((m, 1)), X], axis=1)
```

```
def normalEqn(X, y):

    theta = np.zeros(X.shape[1])

    theta = np.dot(np.dot(np.linalg.inv(np.dot(X.T,X)),X.T),y)

    return theta
```

```
# Calcula los parametros con la ecuación de la normal
theta = normalEqn(X, y);

# Muestra los resultados obtenidos a partir de la aplicación de la ecuación d
print('Theta calculado a partir de la ecuación de la normal: {:s}'.format(str

# Estimar el precio para una casa de superficie de 1650 sq-ft y tres dormitorios

X_array = [1, 20, 400]
price = np.dot(X_array, theta)

print('Capacidad adquisitiva para una edad de 20 años(usando la ecuación de 1
```

```
plotData(X[:, 1], y)
#X = np.concatenate([np.ones((m, 1)), X], axis=1)
#X = np.concatenate([X, X * X], axis=1)

pyplot.plot(X[:, 1], np.dot(X, theta), '-')
```